

Python

08

Локальна область видимості

Локальна область видимості найчастіше використовується в Пайтоні. Коли ми створюємо змінну в блоці коду, її буде дозволено за допомогою найближчої області видимості, або областей. Групування всіх цих областей відоме як середовище блоків коду. Іншими словами, всі призначення виконуються в локальній області видимості за промовчанням.

Локальна область видимості

```
1 def my_func(a, b):  
2     print(a)  
3     print(z)  
4 my_func(1, 2)
```

```
1 Output:  
2 1  
3 NameError: name 'z' is not defined
```

Тут ми створюємо просту функцію, яка приймає два аргументи, далі вона виводить **a** і **z**.

Слід звернути увагу, що ми не визначали змінну **z**, тому коли ми викликаємо функцію ми отримуємо помилку **NameError**. Це відбувається у зв'язку з тим, що змінна не визначена (або не знаходиться в зоні видимості). Якщо ми визначимо змінну перед викликом функції, вона буде знайдена і помилки не буде.

Також, помилка виникне, якщо ми спробуємо отримати доступ до змінної з іншої функції

Глобальна область видимості

Python містить оператор **global**. Цей оператор використовується в тих випадках, коли є необхідність змінити глобальну змінну в локальній функції.

Глобальна область видимості

```
1 x = 10
2
3 def func():
4     print(x)
5     global x
6     x = 20
7     print(x)
8
9 func()
10
11 Output:
12 10
13 20
```

Спочатку **x** має значення, що дорівнює **10**.

Замість того, щоб визначити нову локальну змінну, за допомогою оператора **global**, ми можемо перевизначити змінну **x**.

Однак необхідно мати на увазі, що при зміні значення **x** в середині функції, її значення буде змінено не тільки в області видимості цієї функції, а і в глобальній області видимості. І якщо роздрукувати значення змінної **x** у глобальній області видимості після виклику функції, то її значенням буде вже число **20**, а не **10**.

Варіанти виклику функцій з аргументами

Аргументи за замовчуванням

```
def my_format(string, char='!'):
    result_string = f'{char}{string}{char}'
    return result_string
```

```
my_format('Строка')
```

Збір аргументів до колекції

```
def print_list(*args):
    print(args)
```

```
print_list(1,2,3,4)
```

Узагальнене визначення функції

```
def full_func(*args, **kwargs):
    print(args)
    print(kwargs)
```

```
full_func(1,2,3,a=4,b=5,c=6)
```

Іменовані аргументи

```
def my_pow(number, power):
    result = number ** power + 1
    return result
```

```
result = my_pow(power=3, number=5)
print(result)
```

Збір аргументів у словнику

```
def my_func(**kwargs):
    keys, values = [], []
    for key, value in kwargs.items():
        print(key, value)
        keys.append(key)
        values.append(value)
    return keys, values
```

```
Keys, values = my_func(a=5)
```